Herencia

Programación Orientada a Objetos

Ing. Andrés Eduardo Paternina Ariza Esp.

Universidad del Magdalena

apaternina@unimagdalena.edu.co

¿Dónde estábamos?

En las sesiones anteriores construimos:

- **Definición de Clase y Objeto**
- **Abstracción**
- **Encapsulamiento**

Los cuatro pilares de la POO

Abstracción

Quedarse con lo esencial del dominio y dejar fuera el resto.

Encapsulamiento

Proteger los datos del objeto y permitir su acceso solo a través de sus métodos.

Herencia

Polimorfismo

Enunciado

El hotel **Costa Dorada** necesita desarrollar un sistema para administrar la información de sus trabajadores. Actualmente cuenta con tres tipos de empleados: **recepcionistas, personal de limpieza y gerentes.**

De cada trabajador se necesita almacenar su **número de identificación, nombre completo, salario básico y fecha de contratación.** Sin embargo, cada tipo de trabajador tiene características y responsabilidades particulares.

Recepcionista

- Tiene asignado un idioma principal de atención.
- Atiende huésped

Personal de limpieza

- Tiene asignada una zona del hotel.
- Limpia Habitación

Gerente

- Tiene asignado un departamento.
- Autorizar Gasto

Todos los trabajadores deben poder **consultar su información básica** y conocer su **antigüedad en años** dentro del hotel.

Primera parte: el problema

Recepcionista
Identificación
Nombre
Salario
Fecha contratación
Idioma
Huéspedes atendidos

PersonalLimpieza
Identificación
Nombre
Salario
Fecha contratación
Zona
Habitaciones limpiadas

Gerente
Identificación
Nombre
Salario
Fecha contratación
Departamento
Presupuesto

Primera parte: el problema

Recepcionista	PersonalLimpieza	Gerente
Identificación	**Identificación**	**Identificación**
Nombre	**Nombre**	**Nombre**
Salario	**Salario**	**Salario**
Fecha contratación	**Fecha contratación**	**Fecha contratación**
Idioma	Zona	Departamento
Huéspedes atendidos	Habitaciones limpiadas	Presupuesto

Primera parte: el problema

Recepcionista	PersonalLimpieza	Gerente
Consultar información()	Consultar información()	Consultar información()
Calcular antigüedad()	Calcular antigüedad()	Calcular antigüedad()
Atender huésped()	Limpiar habitación()	Autorizar gasto()

Primera parte: el problema

Recepcionista	PersonalLimpieza	Gerente
Consultar información()	**Consultar información()**	**Consultar información()**
Calcular antigüedad()	**Calcular antigüedad()**	**Calcular antigüedad()**
Atender huésped()	Limpiar habitación()	Autorizar gasto()

¿Qué es herencia?

Una clase **adquiere los atributos y métodos** de otra ya existente, **sin volver a escribirlos**.

Clase padre (superclase)

Es la clase que cede lo que tiene: sus atributos y métodos.

Clase hija (subclase)

Es la clase que lo recibe y puede agregar lo suyo.

Clase padre: Empleado

```
public class Empleado
{
    private string _identificacion;

    public string Identificacion
    {
        get { return _identificacion; }
        private set
        {
            if (string.IsNullOrWhiteSpace(value))
                throw new Exception("Identificación vacía");
            _identificacion = value;
        }
    }

    public Empleado(string identificacion)
    {
        Identificacion = identificacion;
    }

    public void ConsultarInformacion() { }
    public void CalcularAntiguedad() { }
}
```

Clase hija: Recepcionista

```
public class Recepcionista : Empleado
{
    private string _idioma;
    public string Idioma
    {
        get { return _idioma; }
        private set { _idioma = value; }
    }

    public Recepcionista(string identificacion, string idioma)
        : base(identificacion)
    {
        Idioma = idioma;
    }

    public void AtenderHuesped() { }
}
```

Clase hija: PersonalLimpieza

```
public class PersonalLimpieza : Empleado
{
    private string _zona;
    public string Zona
    {
        get { return _zona; }
        private set { _zona = value; }
    }

    public PersonalLimpieza(string identificacion, string zona)
        : base(identificacion)
    {
        Zona = zona;
    }

    public void LimpiarHabitacion() { }
}
```

Clase hija: Gerente

```
public class Gerente : Empleado
{
    private string _departamento;
    public string Departamento
    {
        get { return _departamento; }
        private set { _departamento = value; }
    }

    public Gerente(string identificacion, string departamento)
        : base(identificacion)
    {
        Departamento = departamento;
    }

    public void AutorizarGasto() { }
}
```

¿Cuándo sí usar herencia?

✓ Sí, usa herencia

- Hay una clase **general** y otras que son **variantes especializadas**.
- Las variantes **comparten datos y comportamiento** con la general.
- Puedes decir: **"Un Vendedor** es un **Empleado"** sin sentirte raro.
- Las hijas **redefinen algo concreto** (un cálculo, una regla).

✗ Mejor no

- Solo quieres **reusar código** sin parentesco real entre los conceptos.
- La hija **no es** un tipo del padre — solo lo **usa** por dentro.

La regla del “ES UN”

Lee la relación en voz alta. Si suena natural, la herencia encaja.

Vendedor — *ES UN* — **Empleado**

✓ Suena natural

- **Repartidor** *es un* **Empleado**
- **Estudiante** *es una* **Persona**
- **Auto** *es un* **Vehículo**

✗ Suena forzado

- ~~**Motor** *es un* **Auto**~~
- ~~**Nómina** *es un* **Empleado**~~
- ~~**Factura** *es un* **Cliente**~~

Si “ES UN” Empleado, cabe donde va un Empleado

Una variable Empleado recibe cualquier hija

```
Empleado empleado;
empleado = new Recepcionista("1001", "Inglés");
empleado = new PersonalLimpieza("1002", "Piso 3");
empleado = new Gerente("1003", "Finanzas");
```

Un arreglo Empleado[] guarda los tres tipos

```
Empleado[] empleados = new Empleado[3];
empleados[0] = new Recepcionista("1004", "Francés");
empleados[1] = new PersonalLimpieza("1005", "Lobby");
empleados[2] = new Gerente("1006", "Ventas");

foreach (Empleado e in empleados)
    e.ConsultarInformacion();
```

Enunciado

El hotel **Costa Dorada** necesita desarrollar un sistema para administrar la información de sus trabajadores. Actualmente cuenta con tres tipos de empleados: **recepcionistas, personal de limpieza y gerentes.**

De cada trabajador se necesita almacenar su **número de identificación, nombre completo, salario básico y fecha de contratación.** Sin embargo, cada tipo de trabajador tiene características y responsabilidades particulares.

Recepcionista

- Tiene asignado un idioma principal de atención.
- Atiende huésped

Personal de limpieza

- Tiene asignada una zona del hotel.
- Limpia Habitación

Gerente

- Tiene asignado un departamento.
- Autorizar Gasto

Todos los trabajadores deben poder **consultar su información básica** y conocer su **antigüedad en años** dentro del hotel.

Actividad

Implementa las tres clases derivadas cumpliendo estas reglas:

- Ningún trabajador puede tener un salario negativo.
- La cantidad de huéspedes atendidos y habitaciones limpiadas no puede modificarse directamente.
- Un gerente no puede autorizar gastos superiores al presupuesto disponible.
- La fecha de contratación no puede ser posterior a la fecha actual.
- Las tres clases deben reutilizar los atributos y operaciones comunes mediante herencia.

Para reflexionar

¿Qué pasaría si mañana el hotel incorpora un nuevo tipo de empleado llamado Chef?

¿Qué clases tendrías que modificar y cuáles no?